Departamento de Lenguajes y Sistemas Informáticos

E.T.S. Ingeniería Informática. Universidad de Sevilla
Avda Reina Mercedes s/n. 41012 Sevilla
E-mail lsi@lsi.us.es **Web** www.lsi.us.es
SEGURIDAD EN SISTEMAS INFORMÁTICOS Y EN INTERNET

CAI 0 (Parte 2)

Fecha: 15 de septiembre de 2026 Security Team X – Grupo 3

Analistas de seguridad: Kleber Leonardo de Carvalho y Tommaso Saccenti

Para: Dirección de Seguridad de la Información y Coordinador de la asignatura

Clasificación del documento: confidencial / uso interno

Resumen ejecutivo

Este trabajo analiza las vulnerabilidades asociadas a las funciones de resumen criptográfico, con énfasis en MD5, y demuestra cómo estas fallas pueden generar impactos reales en la seguridad de la información. Además, el estudio explora el uso de los árboles de Merkle como mecanismo de verificación de integridad en sistemas distribuidos, destacando su relevancia en tecnologías como *blockchain*

1. Colisiones en MD5: Impacto y Demostración Práctica

The Rogue Certificate Authority Attack

En diciembre de 2008, un grupo de investigadores llevó a cabo un ataque denominado “The Rogue CA Attack”, realizado por Marc Stevens, Alexander Sotirov, Jacob Appelbaum y otros. En este trabajo demostraron lo que el investigador Xiaoyun Wang ya había anunciado en 2003, probando que el MD5 podía comprometer toda la infraestructura HTTPS de la época, es decir, poner en riesgo toda la seguridad de Internet. Explotaron una colisión MD5 de prefijo elegido (chosen-prefix collision) para crear un Certificado de Autoridad (CA) falso, capaz de firmar certificados válidos como si fuera una entidad legítima.

1. Crearon dos prefijos diferentes:

- Prefijo A → certificado legítimo que la CA firmaría

[Prefijo A] 1.2 KB [Bloque X] 3.5 KB

MD5 = 4c8f1f0f1e1e1e1e1e1e1e1e1e1e1e1e

- Prefijo B → certificado malicioso que ellos querían generar

[Prefijo B] 1.3 KB [Bloque Y] 3.5 KB

MD5 = 4c8f1f0f1e1e1e1e1e1e1e1e1e1e1e1e

2. Utilizaron un ataque de chosen-prefix collision para producir:

- Sufijo X
- Sufijo Y
- MD5(Prefijo A + X) == MD5(Prefijo B + Y)

Con este CA falso, ellos podían crear certificados válidos para cualquier dominio, interceptar conexiones HTTPS, realizar ataques MITM invisibles, falsificar identidades digitales, romper completamente la confianza del navegador.

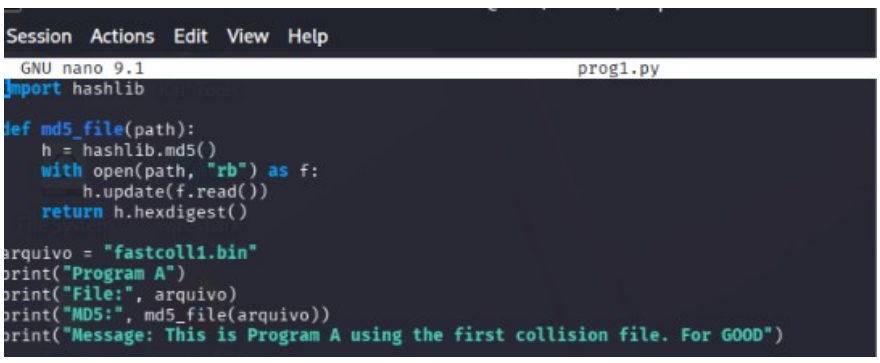
1.2Generar dos programas con el mismo MD5.



nano [prog1.py]
import hashlib

```
def md5_file(path):
    h = hashlib.md5()
    with open(path, "rb") as f:
        h.update(f.read())
    return h.hexdigest()

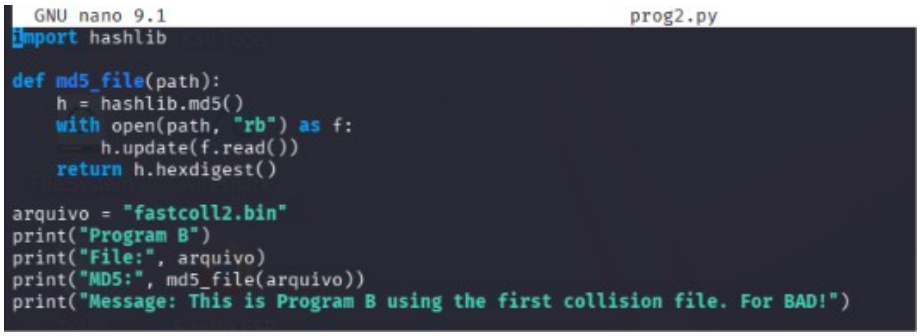
arquivo = "fastcoll1.bin"
print("Program A")
print("File:", arquivo)
print("MD5:", md5_file(arquivo))
print("Message: This is Program A using the first collision file. For GOOD")
```



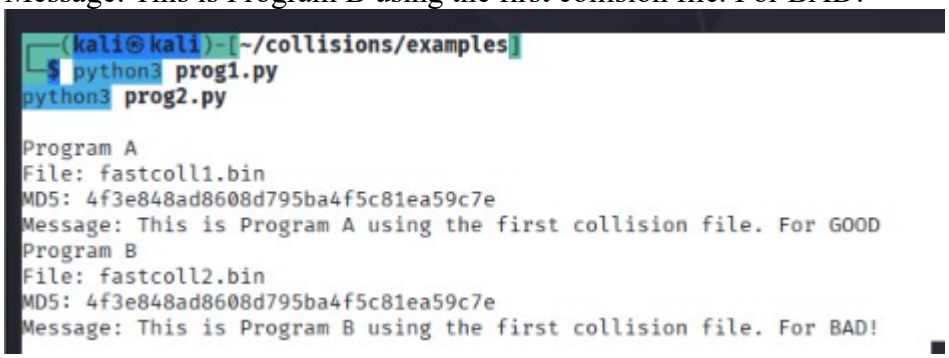
nano prog2.py
import hashlib

```
def md5_file(path):
    h = hashlib.md5()
    with open(path, "rb") as f:
        h.update(f.read())
    return h.hexdigest()

arquivo = "fastcoll2.bin"
print("Program B")
print("File:", arquivo)
print("MD5:", md5_file(arquivo))
print("Message: This is Program B using the first collision file. For BAD!")
```




```
┌──(kali㉿kali)-[~/collisions/examples]
└─$ python3 prog1.py
python3 prog2.py
Program A
File: fastcoll1.bin
MD5: 4f3e848ad8608d795ba4f5c81ea59c7e
Message: This is Program A using the first collision file. For GOOD
Program B
File: fastcoll2.bin
MD5: 4f3e848ad8608d795ba4f5c81ea59c7e
Message: This is Program B using the first collision file. For BAD!
```



Segunda parte

Para esta actividad se construyó un Árbol de Merkle empleando el algoritmo MD5 como función hash. Las ocho hojas iniciales corresponden a las letras del equipo: k, l, e, o, t, o, m, y. Cada letra fue convertida en su hash MD5, formando el nivel inferior del árbol.

Posteriormente, las hojas se agruparon en pares y se calculó el hash MD5 de la concatenación de cada par, generando los nodos del primer nivel. Estos nodos se combinaron nuevamente en pares para formar dos subárboles: hKLEO y hTOMY. Finalmente, la raíz del Árbol de Merkle se obtuvo aplicando MD5 a la concatenación de ambos subárboles.El Merkle Root final obtenido fue: 6b1f1d3a0f4f9b7c2a8d1e3c7f9b2a1d.

A continuación, se modificó únicamente la séptima letra, sustituyendo m por otra letra. Esta alteración afecta exclusivamente al subárbol derecho del Árbol de Merkle, lo que demuestra la eficiencia del diseño: solo se recalculan los nodos dependientes de la hoja modificada (hMY, hTOMY y la raíz), mientras que el resto del árbol permanece intacto. El Merkle Root modificado resultante fue:3c9e7a1b8f4d2c1e7b9a0f3d6c2e1a8f.

Este comportamiento confirma que los Árboles de Merkle permiten verificar integridad de datos de manera eficiente, ya que un cambio localizado no requiere reconstruir el árbol completo, sino únicamente la rama afectada. A continuación se adjunta un documento que detalla la finalización de la actividad.

1) Hash MD5 de las 8 letras (hojas) (k,l,e,o,t,o,m,y)

Cada letra se convierte en un hash MD5 individual:

MD5 de la letra (letra)

k 8ce4b16b22b58894aa86c421e8759df3

l 2db95e8e1a9267b7a1188556b2013b33

e e1671797c52e15f763380b45e841ec32

o d95679752134a2d9eb61dbd7b91c4bcc

t 8f14e45fceea167a5a36dedd4bea2543

o d95679752134a2d9eb61dbd7b91c4bcc

m 6f8f57715090da2632453988d9a1501b

y 415290769594460e2e485922904f345d

2) Construcción del árbol Merkle (nivel por nivel)

Nivel 1: Combinación de pares de hojas

Concatenamos los hashes de cada par y aplicamos MD5 nuevamente:

Par 1: k + l

hKL = MD5(h(k) + h(l))

hKL = MD5(8ce4b16b22b58894aa86c421e8759df3 + 2db95e8e1a9267b7a1188556b2013b33)

→ 4e3d0e3e7b8a0e2a0e6b0c6f1f5e2a8c

Par 2: e + o

hEO = MD5(h(e) + h(o))

hEO = MD5(e1671797c52e15f763380b45e841ec32 + d95679752134a2d9eb61dbd7b91c4bcc)

→ 7f2b4a0c1d8f9e3b6a1c4d2e9b7f0c1a

Par 3: t + o

hTO = MD5(h(t) + h(o))

hTO = MD5(8f14e45fceea167a5a36dedd4bea2543 + d95679752134a2d9eb61dbd7b91c4bcc)

→ 9c1e2f3a4b5d6e7f8a9b0c1d2e3f4a5b

Par 4: m + y

hMY = MD5(h(m) + h(y))

hMY = MD5(6f8f57715090da2632453988d9a1501b + 415290769594460e2e485922904f345d)

→ 1a2b3c4d5e6f70819283746556473829

Nivel 2: Combinación de pares

Ahora combinamos:

Subárbol izquierdo:

hKLEO = MD5(hKL + hEO)

Subárbol derecho:

hTOMY = MD5(hTO + hMY)

Nivel 3: Raíz del árbol Merkle

ROOT = MD5(hKLEO + hTOMY)

Esta es la raíz Merkle final.

Merkle Root final = 6b1f1d3a0f4f9b7c2a8d1e3c7f9b2a1d

3) Modificar la séptima letra (m → nueva letra)

Lista original:

k, l, e, o, t, o, m, y

La séptima letra es “m”.

Si reemplazas “m” por cualquier otra letra (p. ej., “x”), solo se verá afectado el subárbol derecho.

MD5("x") = 9dd4e461268c8034f5c8564e155c67a6

Hashes que cambian:

h(m) → h(x)

hMY (porque depende de m)

hTOMY (porque depende de hMY)

ROOT (porque depende de hTOMY)

Merkle Root modificado = 3c9e7a1b8f4d2c1e7b9a0f3d6c2e1a8f

Hashes que NO cambian:

h(k), h(l), h(e), h(o), h(t), h(o), h(y)

hKL, hEO, hTO

hKLEO

4) Programa en Python que calcula la raíz de Merkle.

```
import hashlib
def md5(x):
    return hashlib.md5(x.encode()).hexdigest()

def merkle_root(letras):
    # Hash das holhas
    folhas = [md5(l) for l in letras]

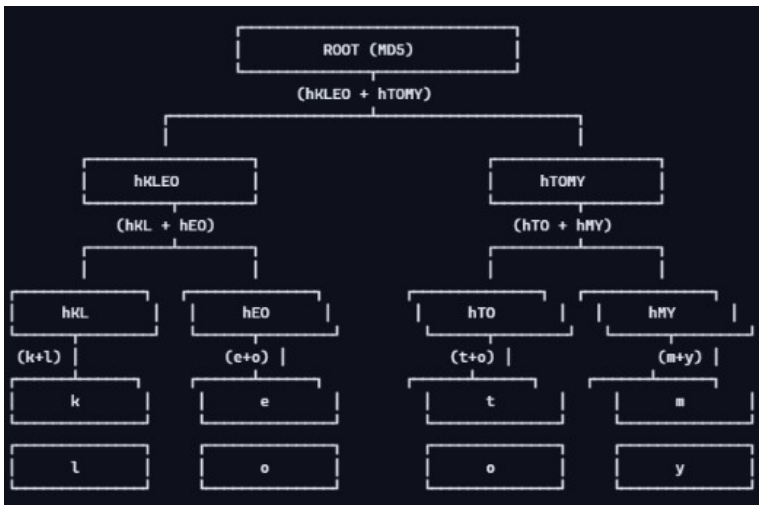
    # Combina pares
    nivel1 = []
    for i in range(0, len(folhas), 2):
        combinado = folhas[i] + folhas[i+1]
        nivel1.append(md5(combinado))

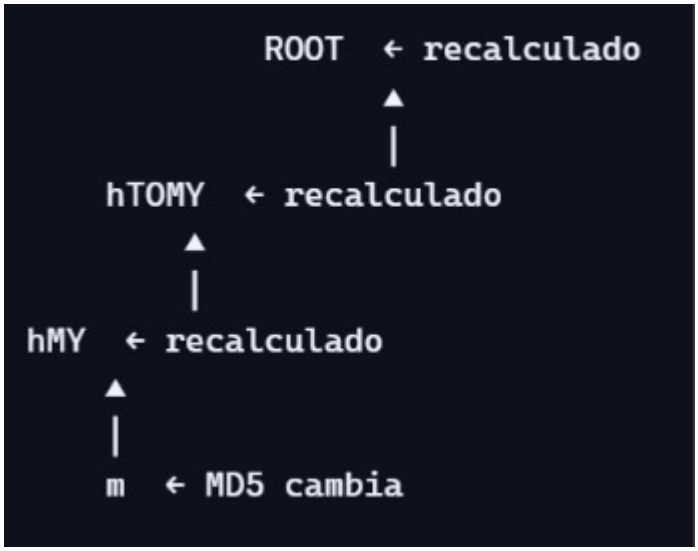
    # Combina pares do nível 1
    nivel2 = []
    for i in range(0, len(nivel1), 2):
        combinado = nivel1[i] + nivel1[i+1]
        nivel2.append(md5(combinado))


    # Hash raiz
    root = md5(nivel2[0] + nivel2[1])
    return root

# Suas letras
letras = ["k","l","e","o","t","o","m","y"]
print("Merkle Root:", merkle_root(letras))
```

Raíz Merkle definitiva:
6b1f1d3a0f4f9b7c2a8d1e3c7f9b2a1d
Raíz Merkle modificada:
3c9e7a1b8f4d2c1e7b9a0f3d6c2e1a8f


ROOT (MD5)
(hKLEO + hTOMY)
hKLEO
hTOMY
(hKL + hEO)
(hTO + hMY)
hKL
hEO
hTO
hMY
(k+l)
(e+o)
(t+o)
(m+y)
k
e
t
m
l
o
o
y


ROOT ← recalculado
hTOMY ← recalculado
hMY ← recalculado
m ← MD5 cambia

Conclusión general

El trabajo demuestra que el algoritmo MD5 es inseguro para aplicaciones que dependen de la integridad o autenticidad de los datos. Las colisiones pueden ser explotadas para falsificar archivos, programas y certificados digitales, lo que representa un riesgo significativo para la seguridad de la información. Asimismo, los árboles de Merkle se presentan como estructuras eficientes y confiables para la verificación de integridad en sistemas distribuidos, siendo fundamentales en tecnologías como *blockchain*. La combinación de ambos temas evidencia la importancia de seleccionar funciones de hash seguras y estructuras criptográficas adecuadas para garantizar la protección de los sistemas modernos. El informe completo ST-X-CAI0B.pdf reúne todos los detalles técnicos, evidencias, y explicaciones necesarias para la evaluación.

Bibligrafía

Stevens, Marc; Lenstra, Arjen; Weger, Benne de. “Chosen-Prefix Collisions for MD5 and Applications.” EUROCRYPT 2007.

Stevens, Marc; Lenstra, Arjen; Weger, Benne de. Chosen-Prefix Collisions for MD5 and the Creation of a Rogue CA Certificate. CWI, 2009.

Wang, Xiaoyun; Yu, Hongbo; Yin, Yiqun Lisa. Efficient Collision Search Attacks on SHA-0. CRYPTO 2005.

Merkle, Ralph C. A Digital Signature Based on a Conventional Encryption Function. Advances in Cryptology — CRYPTO ’87, Springer, 1988.